Cloud Computing

Lecture 2 - Enabling technologies

Emiliano Casalicchio
Department of Computer Science
Sapienza University of Rome

Agenda

- Distributed computing
- Architectural styles
- Service Oriented Architectures
  - Web. Services
  - Microservices

Distributed systems

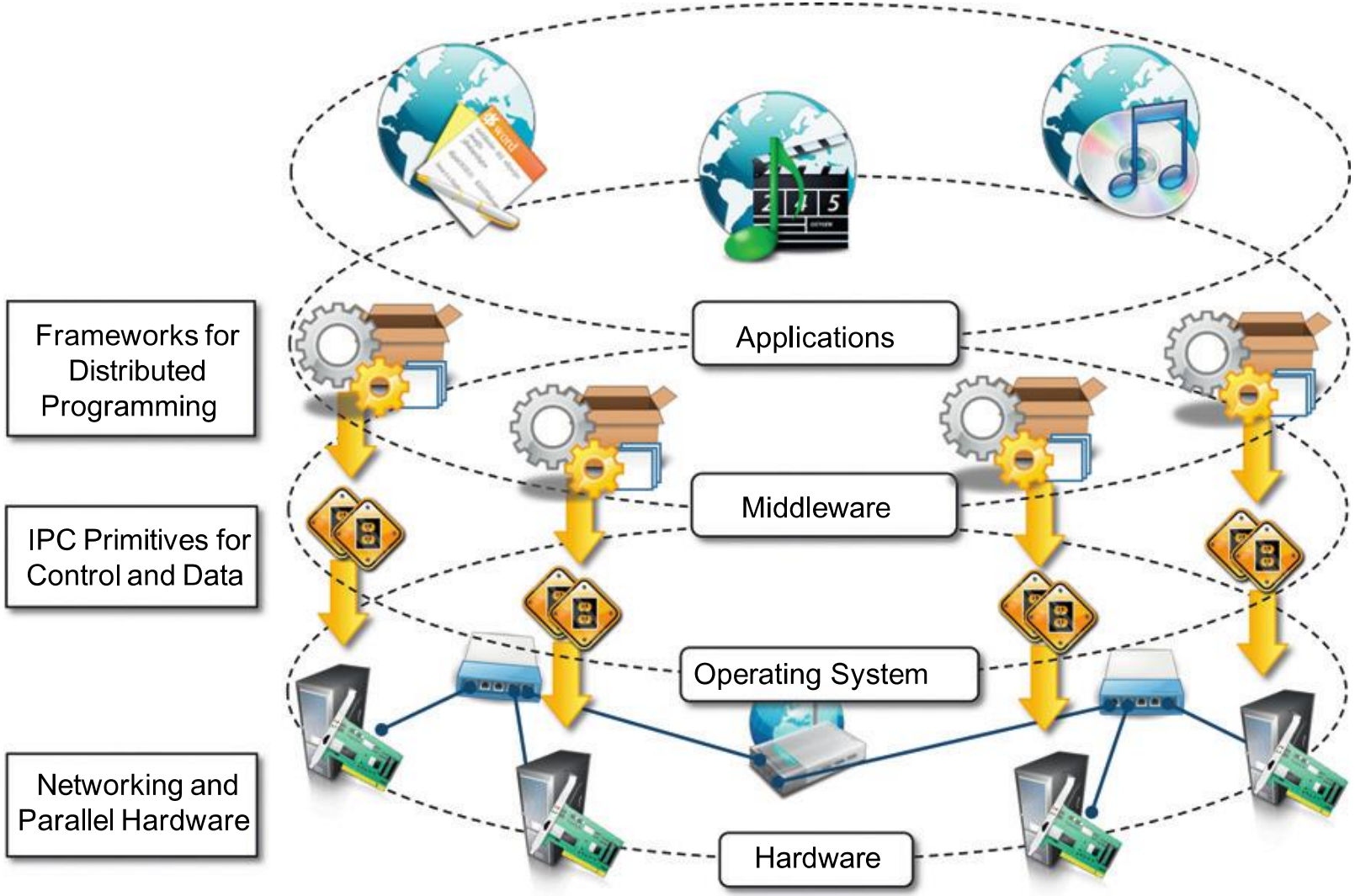
- *A distributed system is a collection of independent computers that appears to its users as a* ***single coherent system*** *[Tanenbaum]*
  - Focus on unified usage and aggregation of distributed resources

- *A distributed system is one in which components* ***located at networked computers*** *communicate and coordinate their actions only by* ***passing messages*** *[Coulouris]*
  - Focus on the networked nature and communication among processes by passing messages

Distributed computing

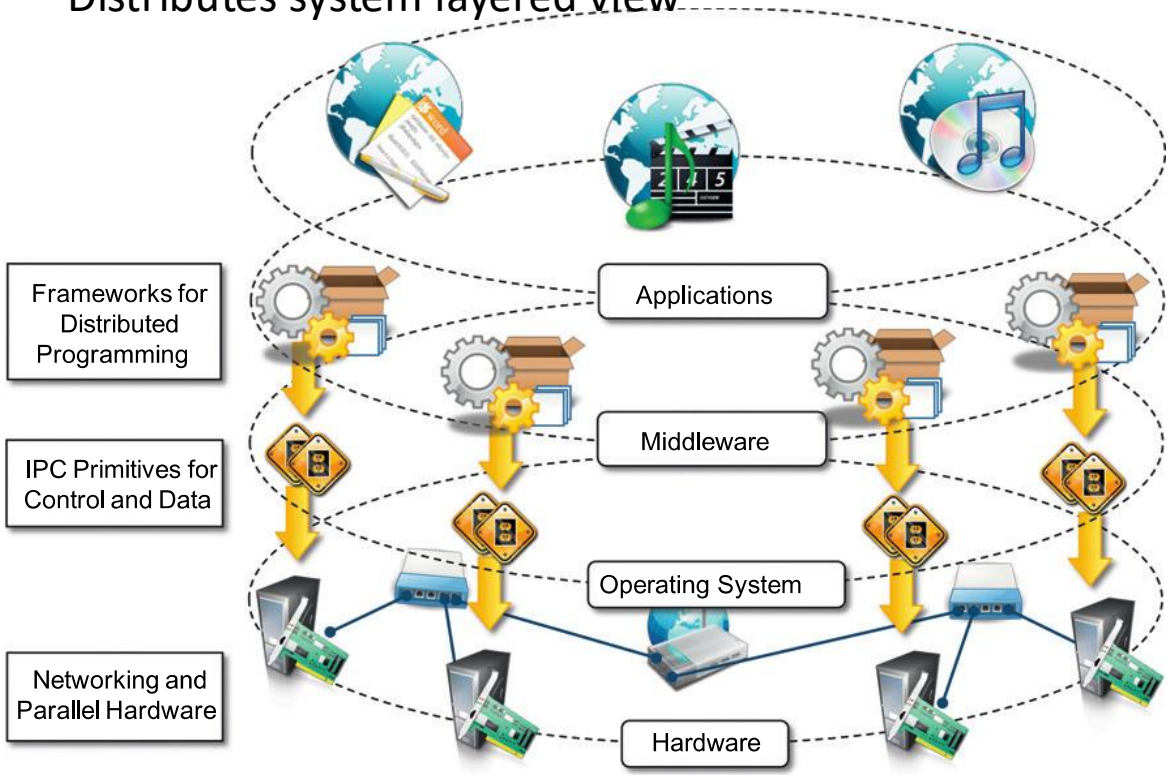
- Is a **computational models; Defines how computation is carried on**
- The computation is broken down into units executed concurrently on different computing elements
  - Processors on different nodes, processors on the same computer, or cores within the same processor.
- Distributed often implies
  - Different locations of the computing elements
  - Heterogeneity in terms of hardware and software features

Components of a distributed system

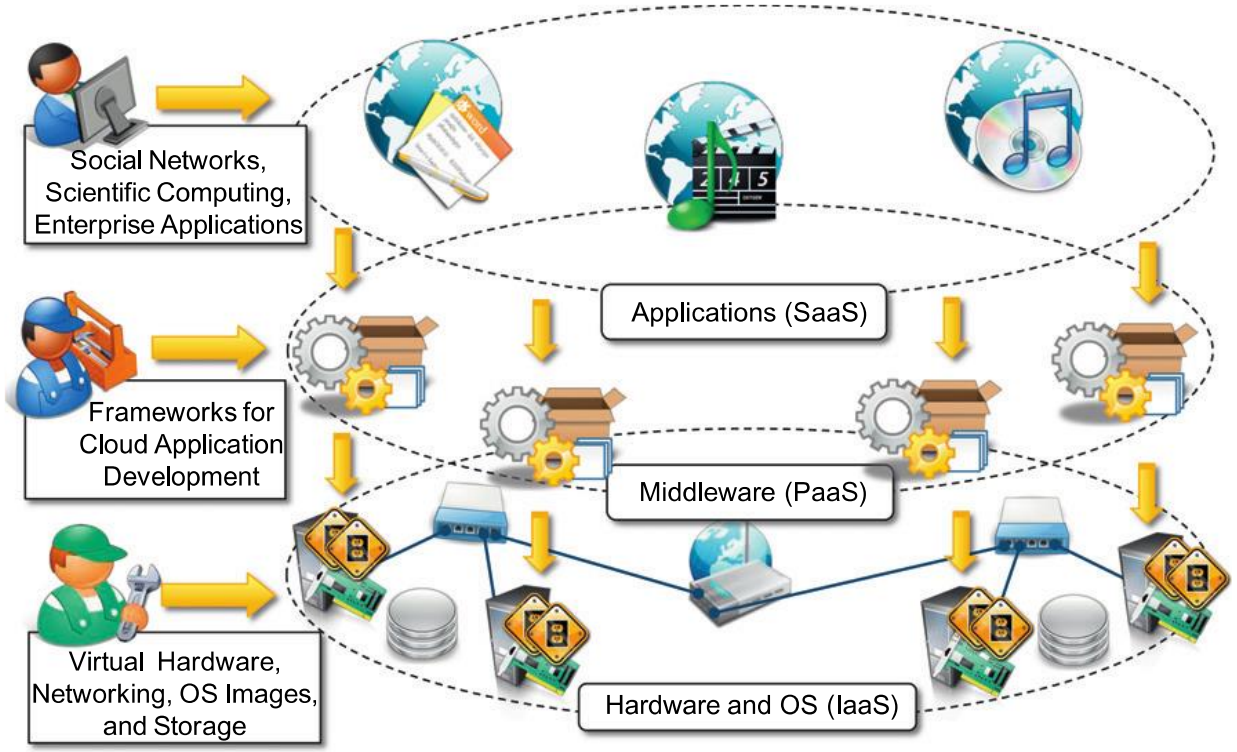


Distributed system and Cloud computing systems

Distributes system layered view



Cloud Computing system layered view



Cloud computing is a specialized form of distributed computing that introduces utilization models for remotely provisioning scalable and measured resources

Architectural styles

- Define
  - the different roles of components in the distributed system
  - how components are distributed across multiple machines
  - what connectors (comm. mechanisms) are used
  - a family of systems in terms of a pattern of structural organization;

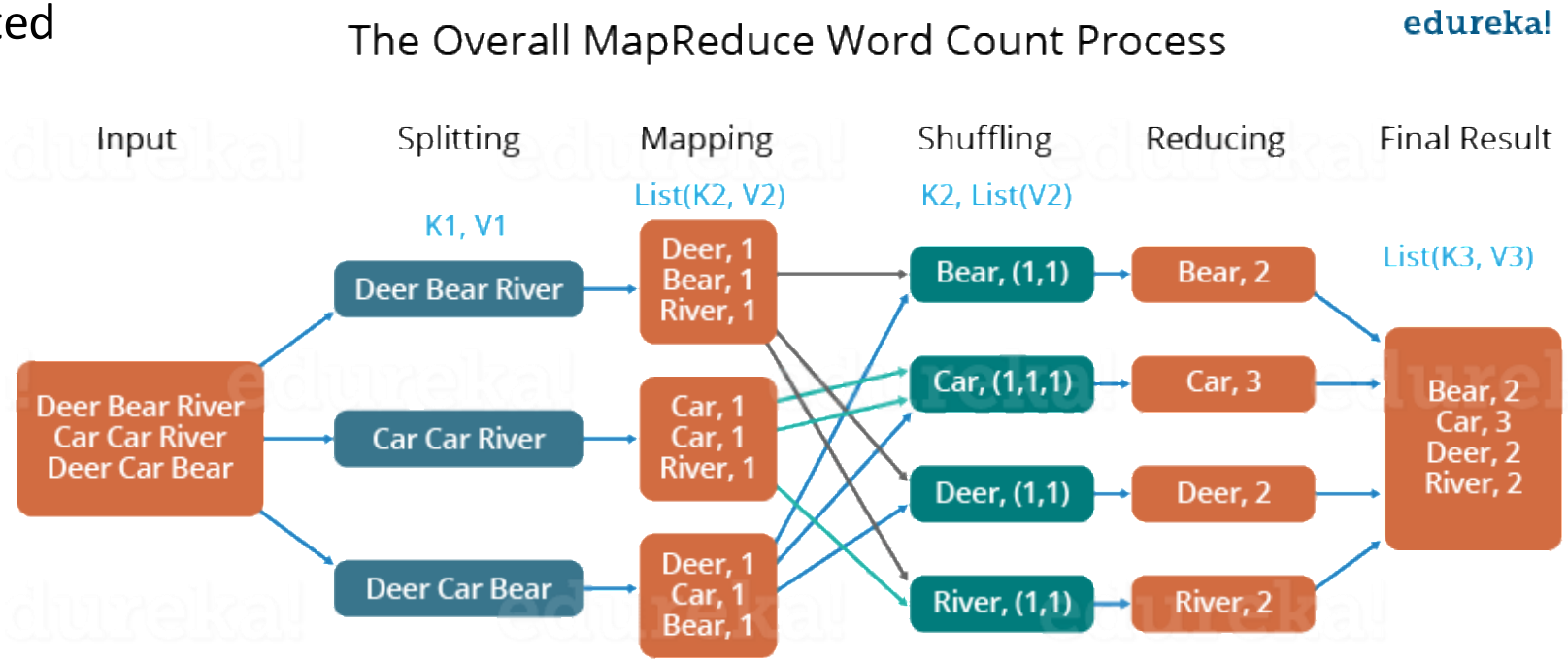
- Are mainly used to determine the vocabulary of components and connectors that are used as instances of the style together with a set of constraints on how they can be combined

Software architectural styles

Category	Most Common Architectural Styles
Data-centered	Repository Blackboard
Data flow	Pipe and filter Batch sequential
Virtual machine	Rule-based system Interpreter
Call and return	Main program and subroutine call/top-down systems Object-oriented systems Layered systems
Independent components	Communicating processes Event systems

An example of pipe and filter

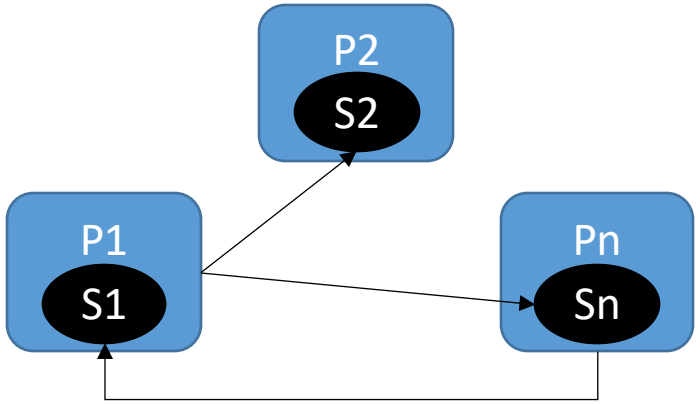
- map reduce paradigm
  - A simple word count can be distributed



Independent Components

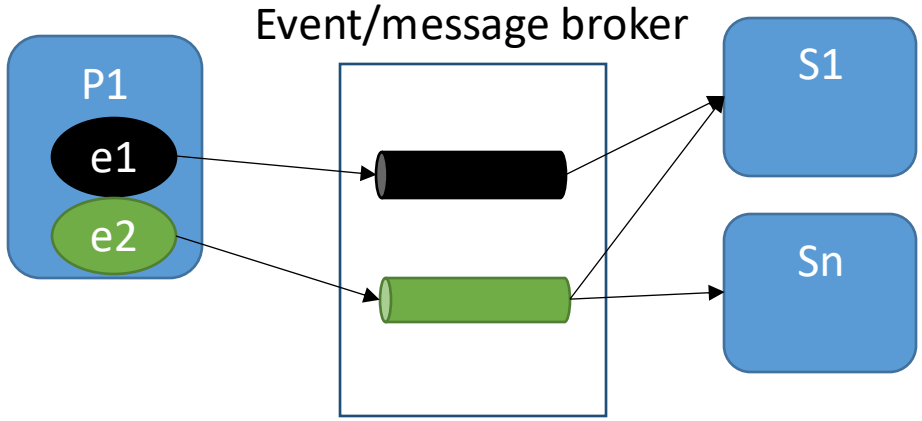
- Components have their own life cycles and interact each other to perform their activities
  - Communicating processes
  - Event systems

- Communicating processes
  - Independent processes using IPC (RPC, Web Services, REST, DO) for coordination management
  - Each process provides services and can use services exposed by the other processes
  - Physically organized in architectures like C/S; P2P; SOA



Independent Components: Event based systems

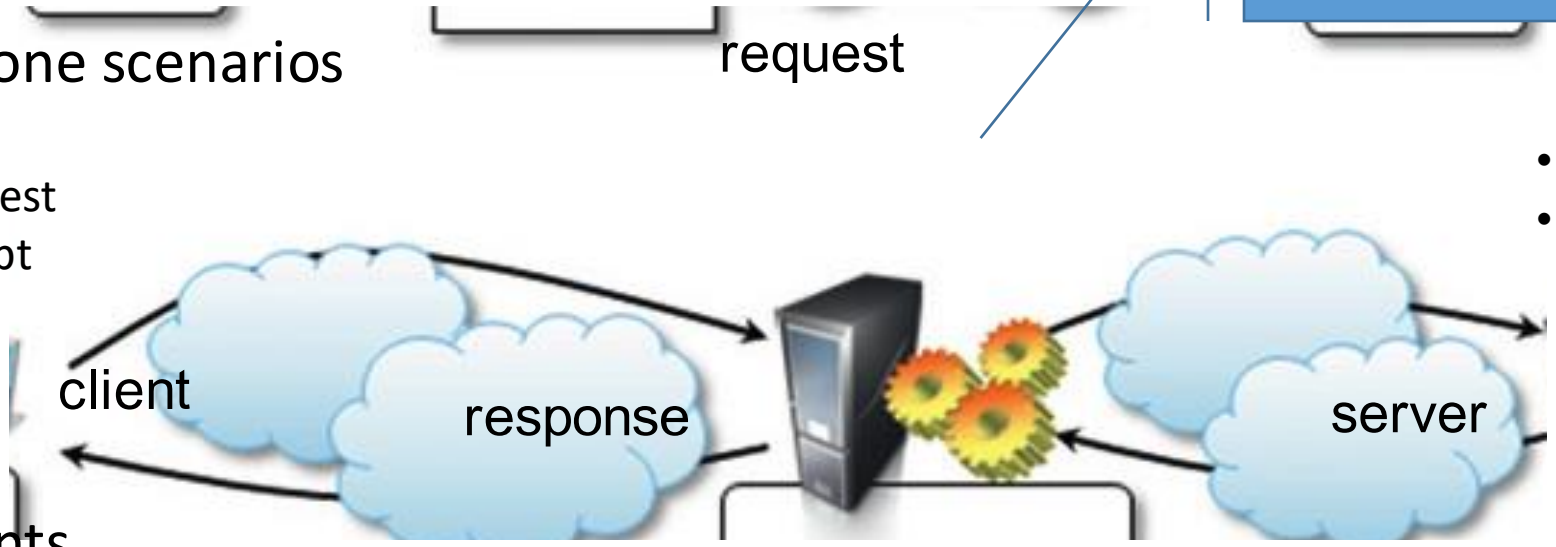
- loosely coupled and connected components
- Each component publish a collection of events other components can subscribe
- Subscriber component provides a callback that will be executed when the event is activated
- Foster the development of open systems: a component can be easily integrated as long as they have compliant interfaces for registering to events



Client/Server architecture style

Communication protocols
HTTP, TLS, FTP

- Suitable for many-to-one scenarios



- Request
- Accept

- Listen
- Response

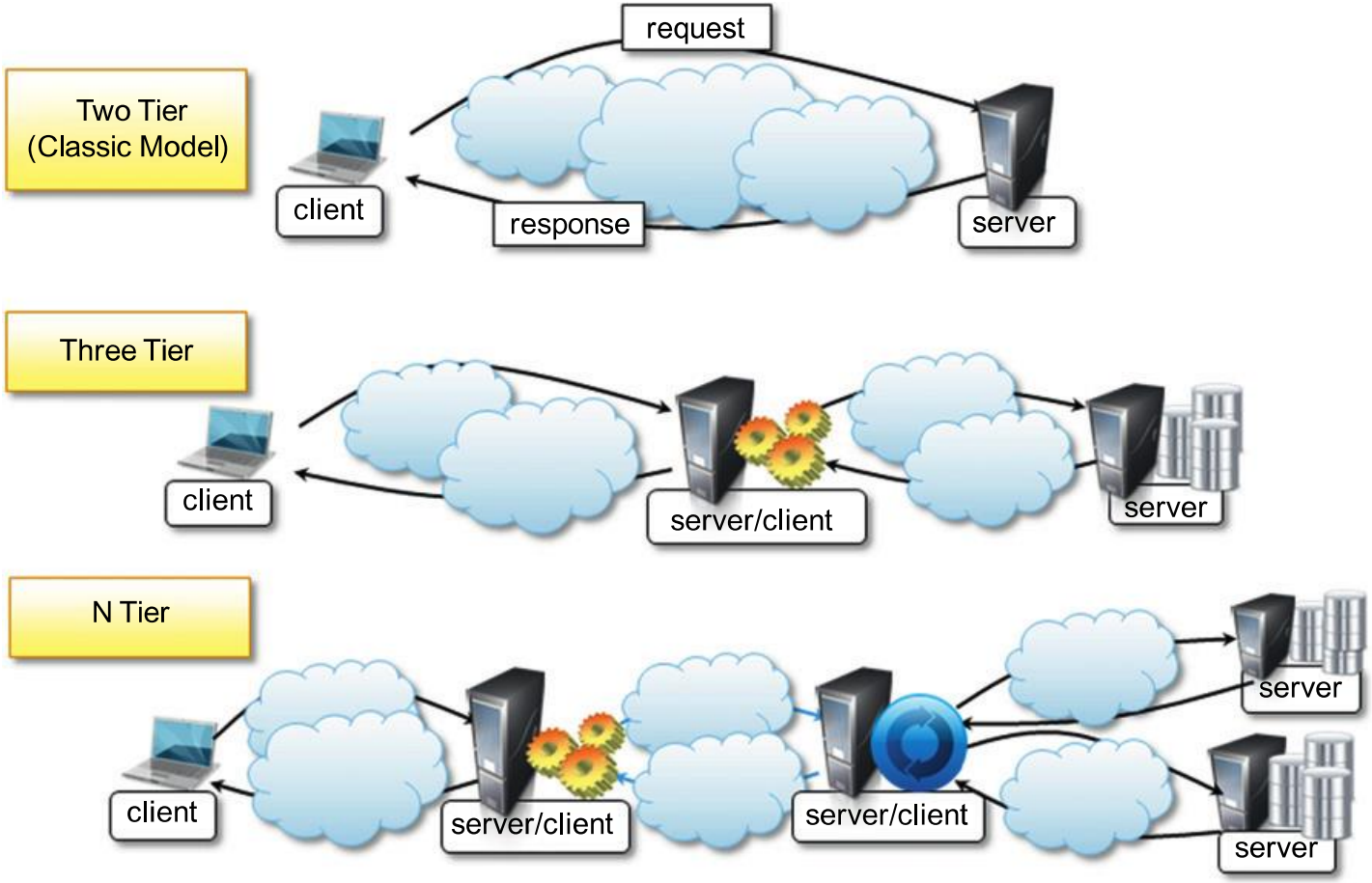
- Three main components
  - presentation, application logic, and data storage
- Thin-client model
  - Server: application logic + data storage; Client: presentation
- Fat-client model
  - Server: data storage; Client: presentation + application logic

C/S architecture style: the multi-tiered model

- Presentation, application logic, and data maintenance can be seen as conceptual layers, which are more appropriately called tiers.

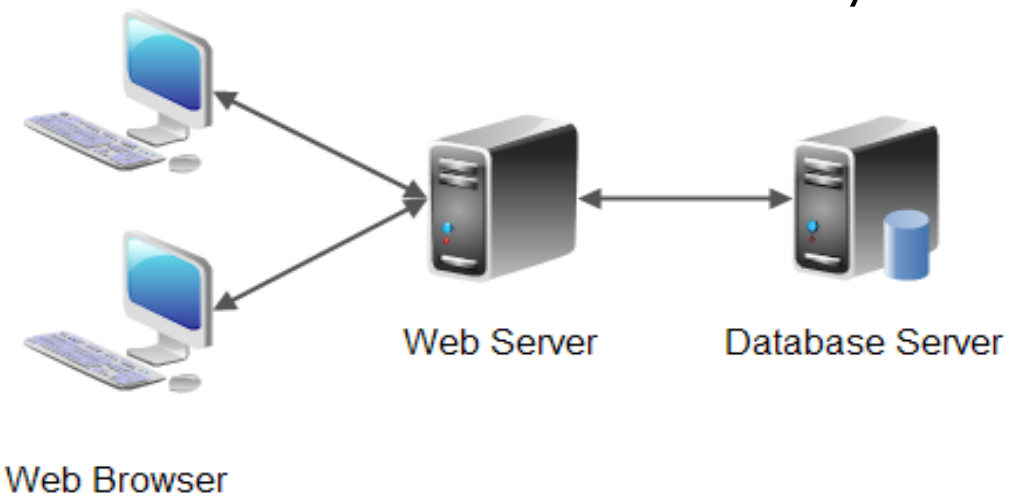
- Suitable for systems of limited size
- Suffers from scalability issues

- More scalable, it is possible to distribute the tiers into several computing nodes;
- performance bottlenecks could be isolated

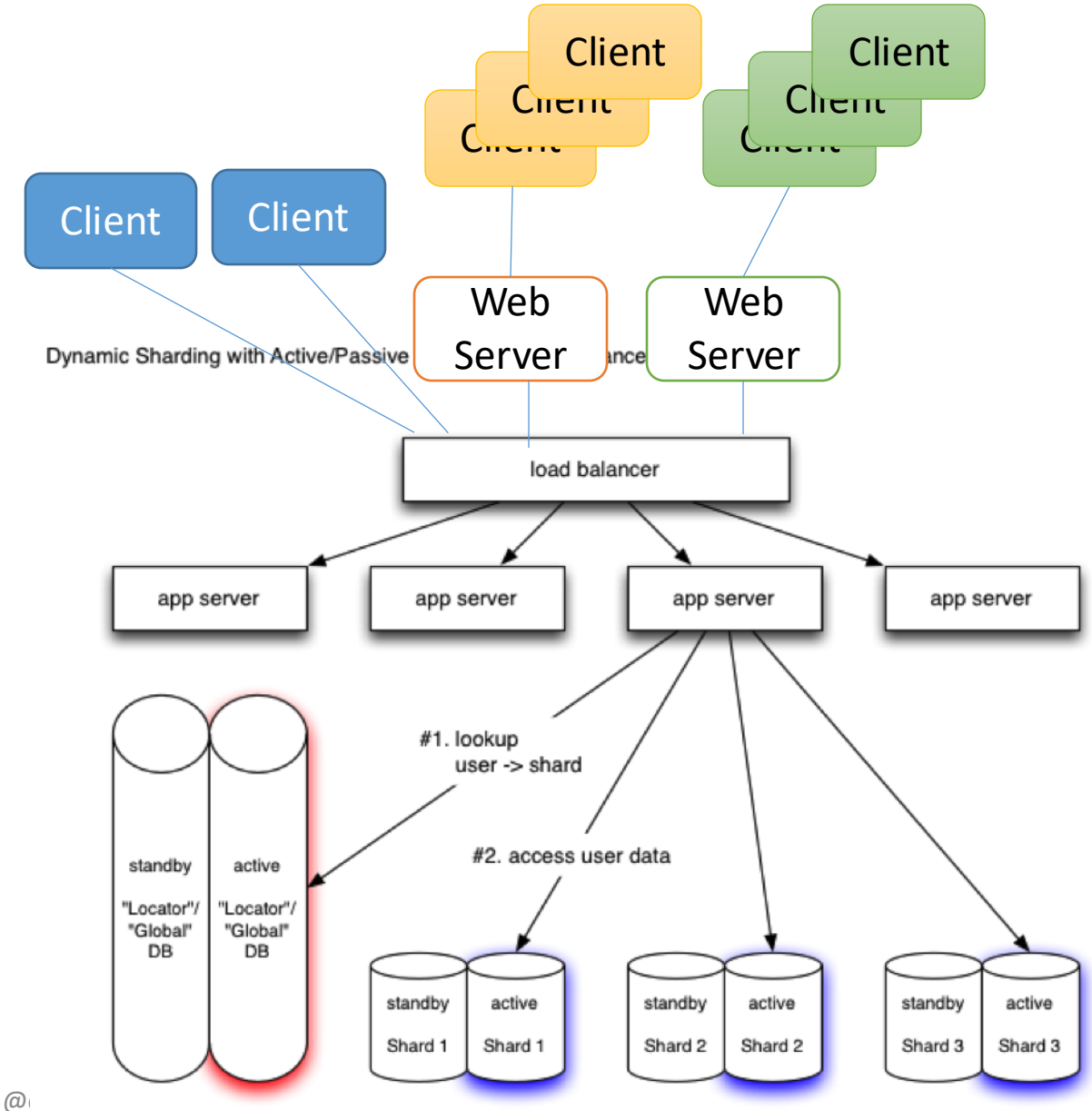


Examples

Three-tier web server system

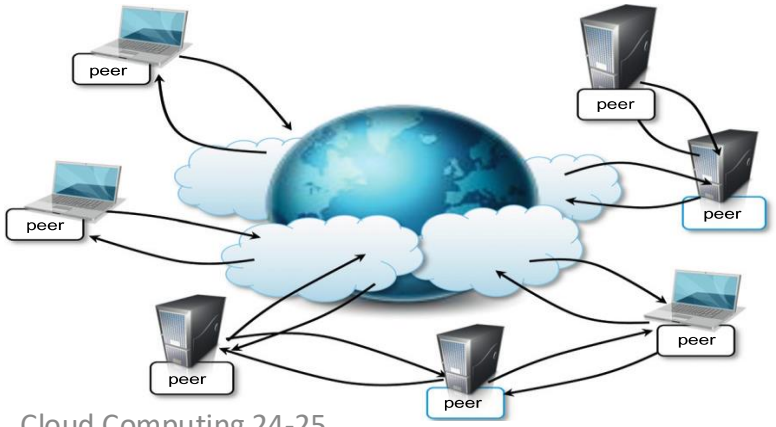


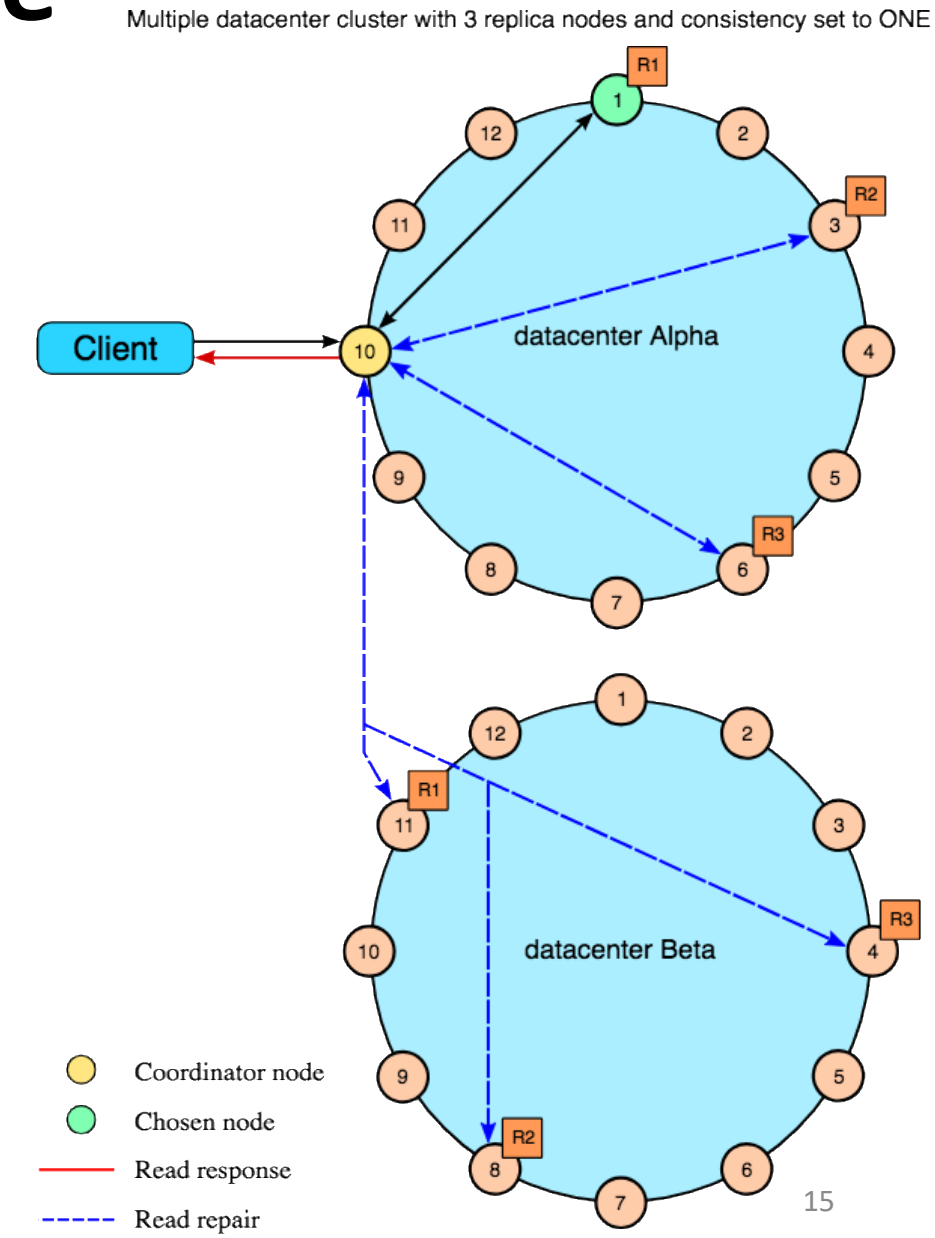
N-tier web server system



Peer-to-peer architecture style

- Symmetric architecture
  - All the components (peers) play the same role and incorporate both client and server capabilities
- Suitable for highly decentralized architecture
  - Can scale better along the dimension of the number of peers
- E.g. file-sharing applications, distributed storage/cloud storage applications





Service orientation

Service Oriented Architecture and Microservices

Service Oriented Architecture (SOA)

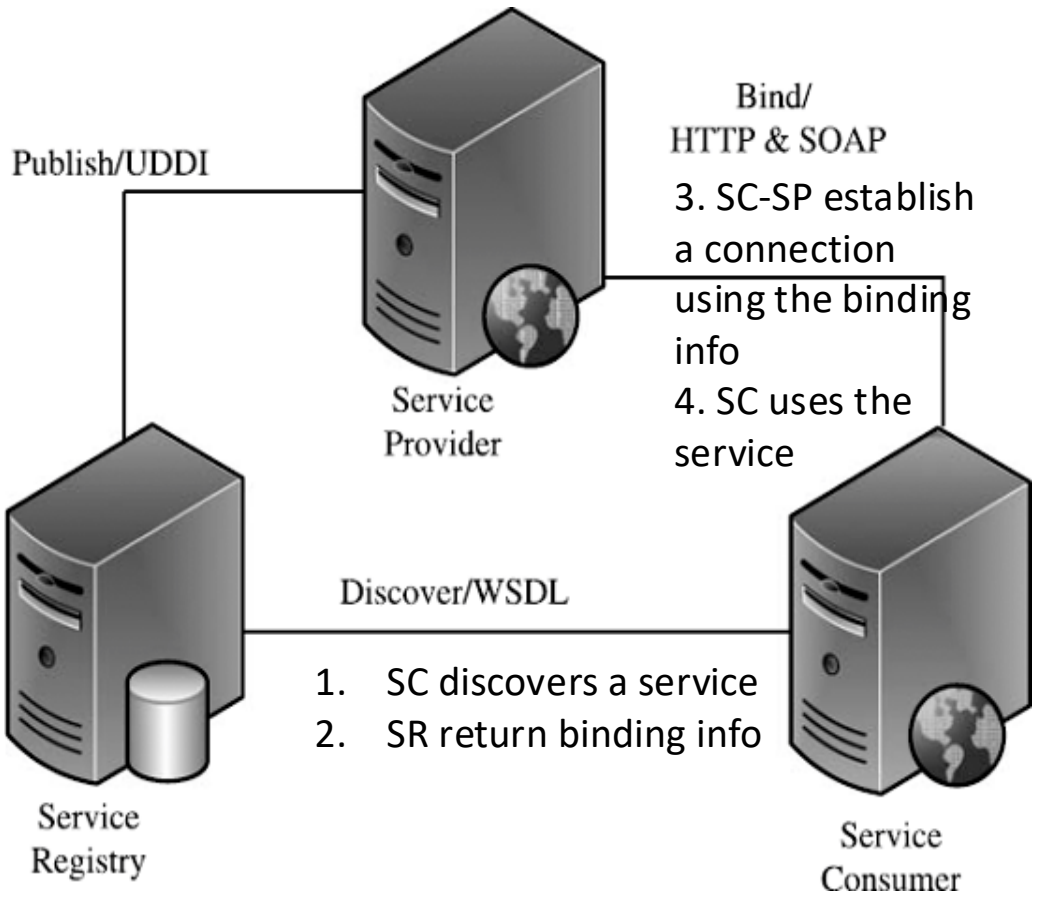
- Is an architectural style that supports service orientation.
- Service orientation is a way of thinking in terms of services and service-based development and the outcomes of services.
- A service:
  - Is a logical representation of a repeatable business activity that has a specified outcome (e.g., check customer credit; provide weather data, consolidate drilling reports)
  - Is self-contained
  - May be composed of other services
  - Is a “black box” to consumers of the service

Service Oriented Architecture

- An application is an aggregations of services that are coordinated within a SOA
- A service encapsulates/is implemented by a software component and has four main characteristics
  - **Boundaries are explicit**, interfaces are kept minimal to foster reuse and simplify interaction
  - **Services are autonomous**, can be integrated in different systems at the same time, can handle failures
  - **Services share schema and contracts**, a contract describe the structure of messages the service can send and/or receive and additional constraint on their ordering
    - allow to check structural compatibility using machine-based techniques
  - **Service (semantic) compatibility is determined based on policies**
    - Policies define the capabilities and requirements in term of expressions that must hold true

Service Oriented Architecture (SOA)

- There are three major roles within SOA
  - Service provider
    - maintainer of the service
    - Publish the service in a registry together with a service schema and contract
  - Service consumer
    - locate the service metadata in the registry
    - develop the required client components to bind and use the service.
  - Service Registry
    - Store the service descriptors
- Service providers and consumers can belong to different organization bodies or business domains.



SOA feature and benefits

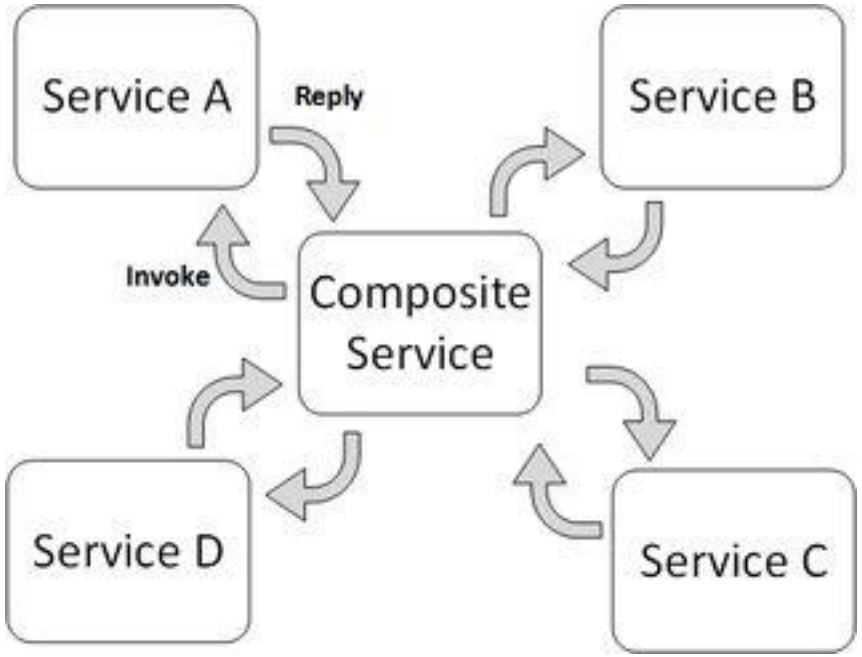
- The most relevant are
  - Service Re-Use
  - Message monitoring, control, transformation
  - Complex event processing
  - Service composition
  - Service Discovery

Service Oriented Architecture (SOA)

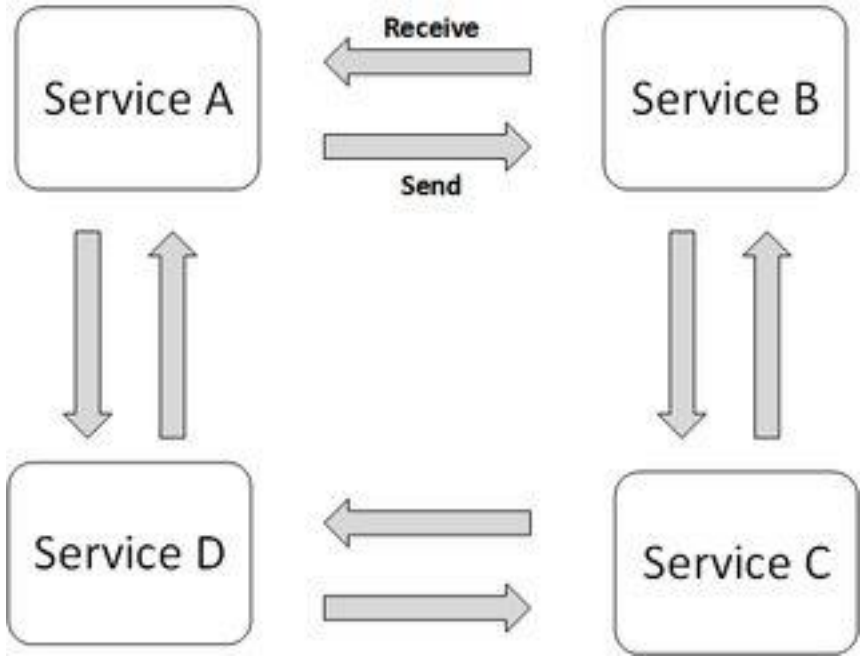
- Components can play the roles of both service provider and service consumer
  - Aggregating services in workflows and using them to satisfy consumer requests
    - E.g. travel planner – search for flight, search for hotel, book hotel, search for car rental/sight-seen attractions, …
- How services can be coordinated and managed?
  - Service Orchestration
    - A service orchestrator coordinate and manage the composition and interaction of services in a centralized way
  - Service choreography
    - Coordinated interaction (message exchange) of services without a single point of control

Orchestration and Choreography

Service orchestration represents a single centralized executable business process (the orchestrator) that coordinates the interaction among different services. The orchestrator is responsible for invoking and combining the services.

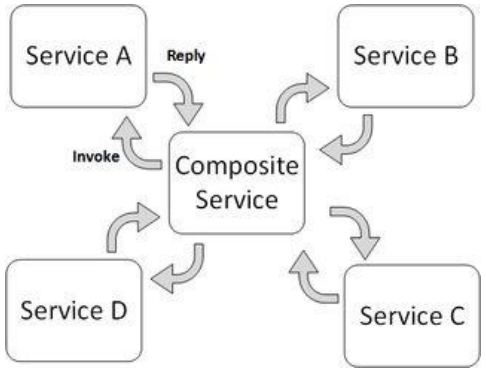


Service Choreography is a global description of the participating services, which is defined by exchange of messages, rules of interaction and agreements between two or more endpoints. Choreography employs a decentralized approach for service composition.



Examples

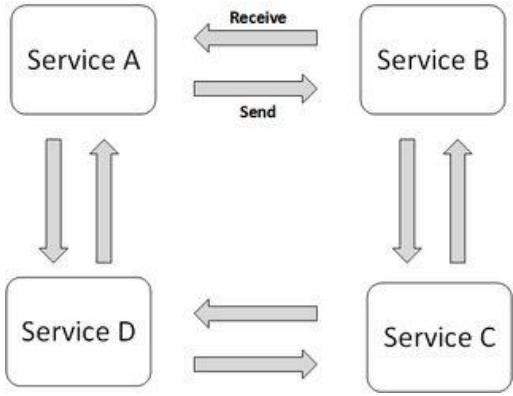
- Orchestration: SW components are part of a workflow runs by a central entity
  - Travel planner
    - select destination, display hotels, select hotel, suggest options (likerent car, buy attractions, …), …
  - Flight/train/event ticket reservation
    - Select dep/arr location, select dates, display options, select flight 1 … n, select ticket class, select options …



Examples

- Choreography: SW components are independent and exchange messages to interact and achieve their goal
  - Online payment with credit card to conclude an online purchase
    - Seller Transaction Service sends a CC transaction request to the Credit Card company Service;
    - Credit Card company service accept/deny and send the answer to the Seller Transaction Service ;

  - Authentication (of systems, services)
    - Client Service send a Auth. Req to the Auth. Service
    - The Auth. Service verify credentials, eventually sent a three factor auth request
    - (eventually) The Client send three factor auth credential
    - The auth service reply to the Client Service granting/denying access



casalicchio@di.uniroma1.it

Microservices

“**Microservices Patterns Chris Richardson**”

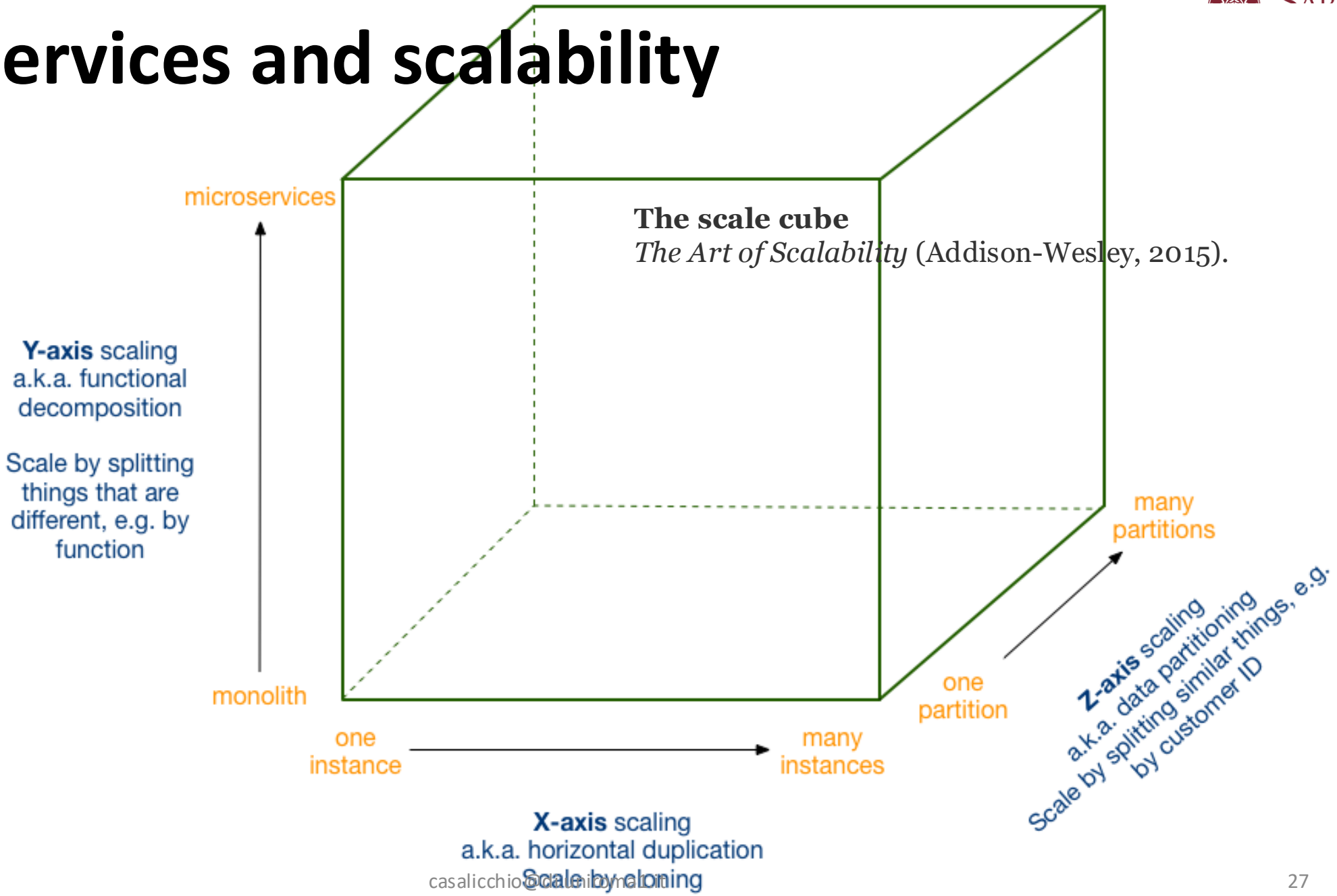
Available online

Chapter 1 of the book

Microservice architecture (o microcervices)

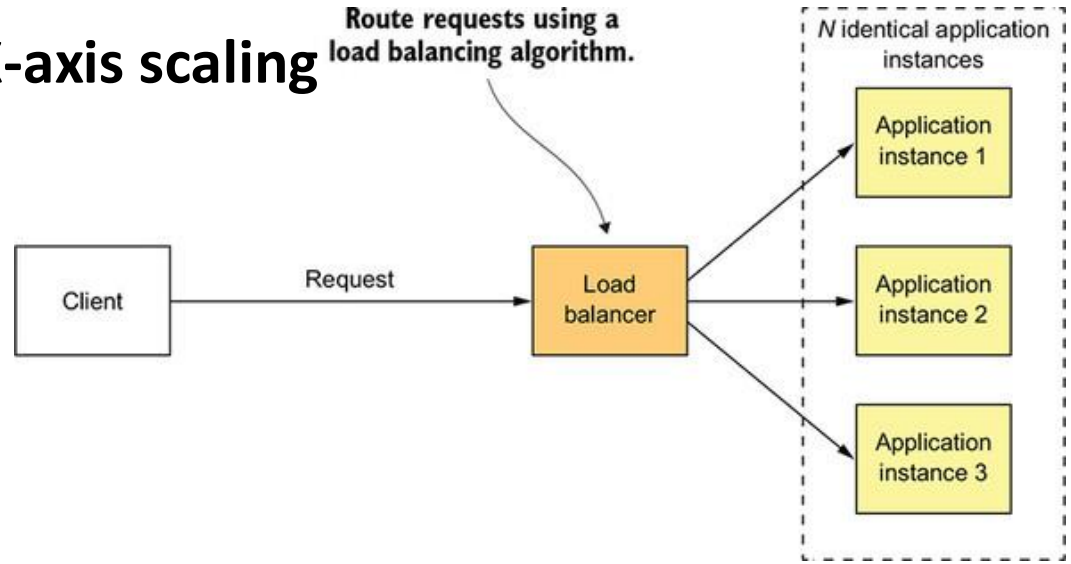
- Is an architectural style that structures an application as a collection of services that are
  - Highly maintainable and testable
  - Loosely coupled
  - Independently deployable
  - Organized around business capabilities
  - Owned by a small team
- The microservice architecture enables
  - the rapid, frequent and reliable delivery of large, complex applications
  - an organization to evolve its technology stack

Microservices and scalability

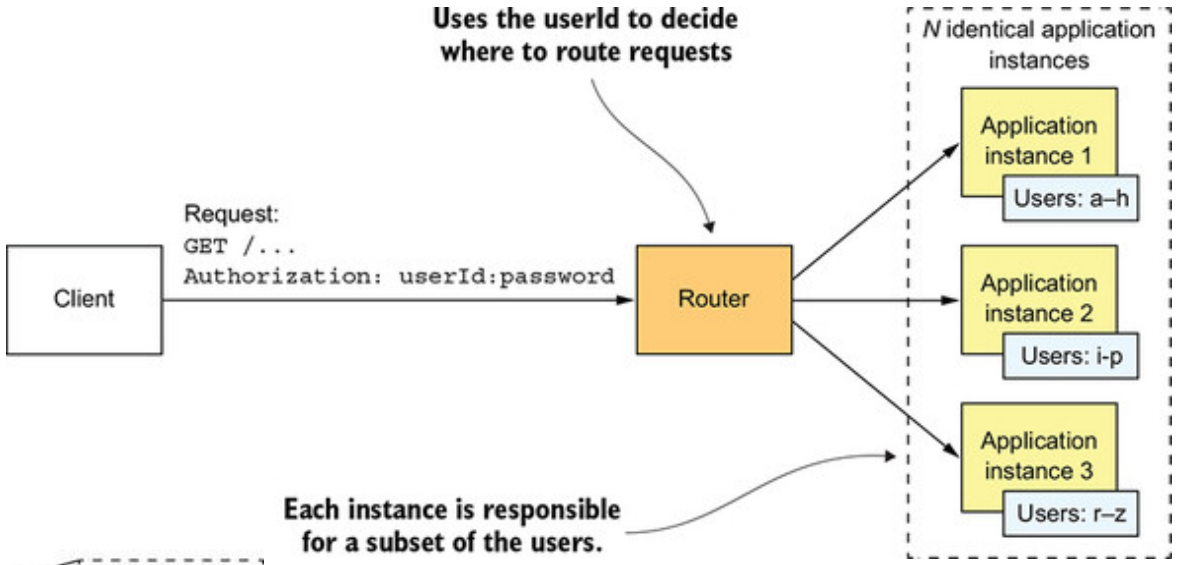


The scale cube
The Art of Scalability (Addison-Wesley, 2015).

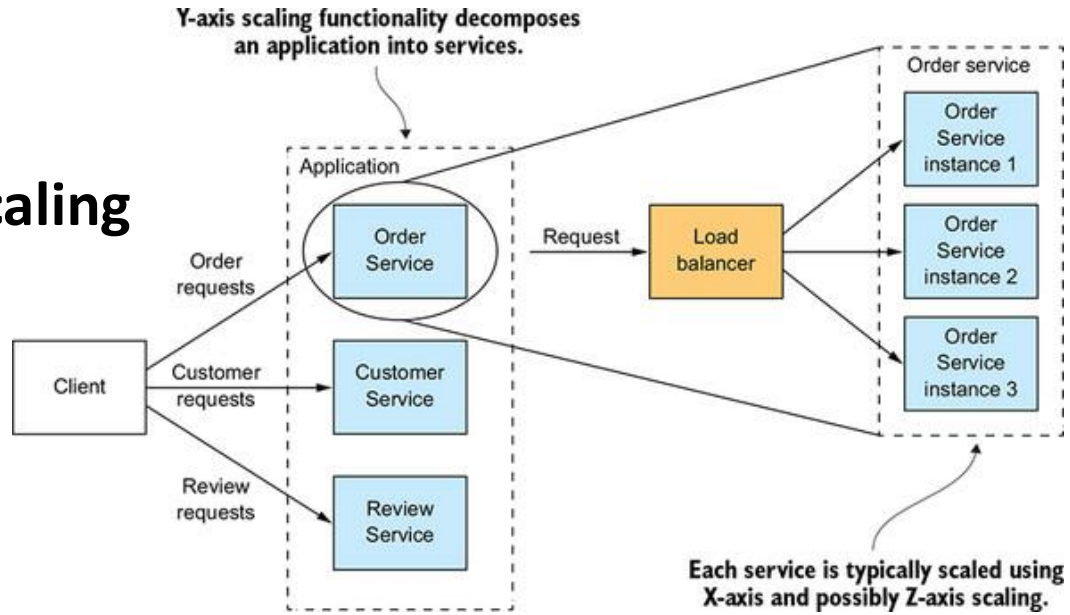
X-axis scaling



Z-axis scaling



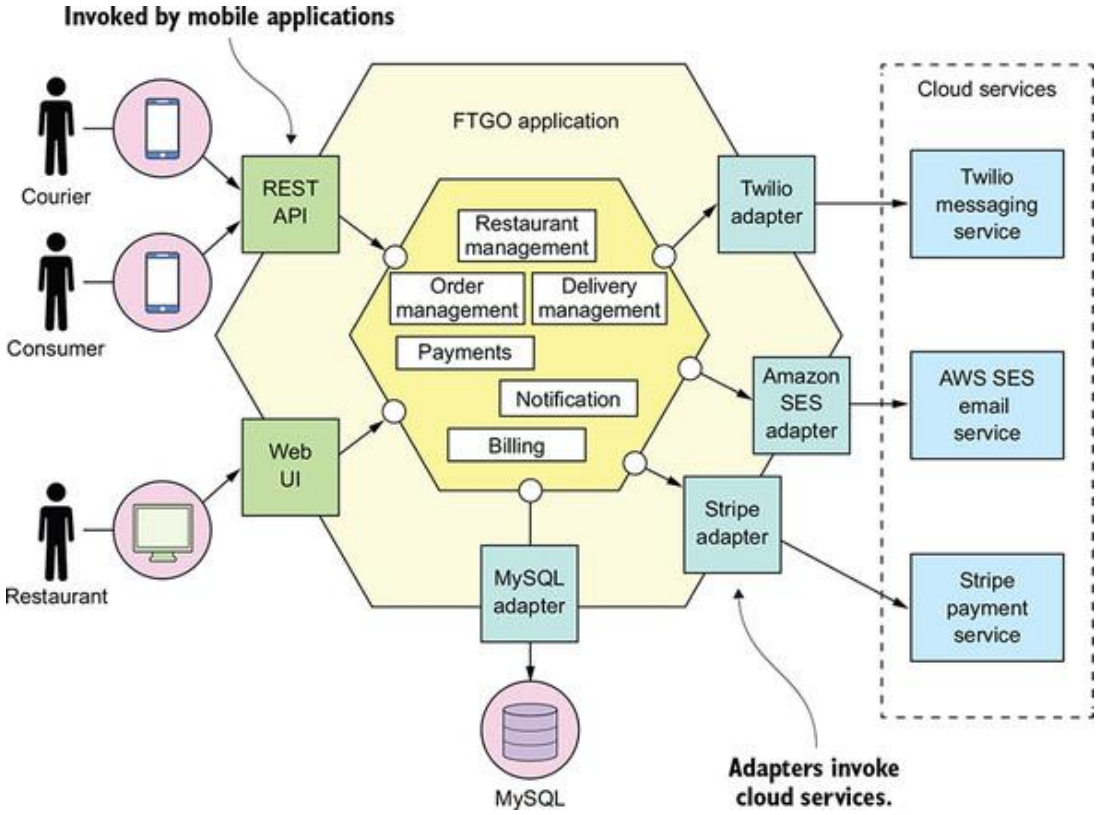
Y-axis scaling



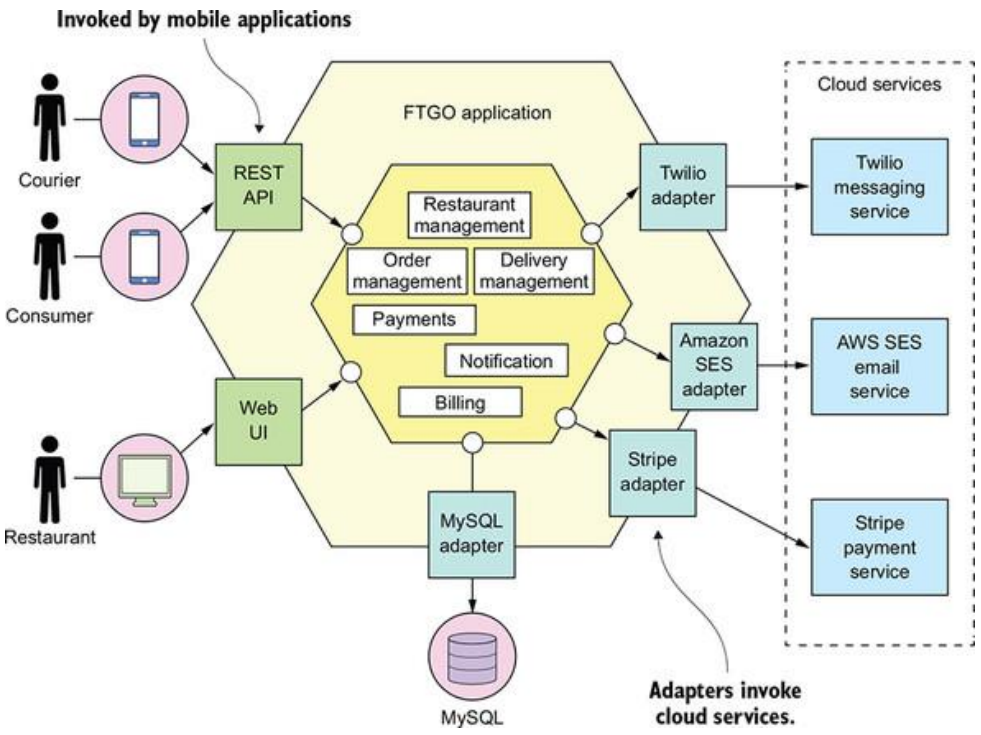
From microservices patterns

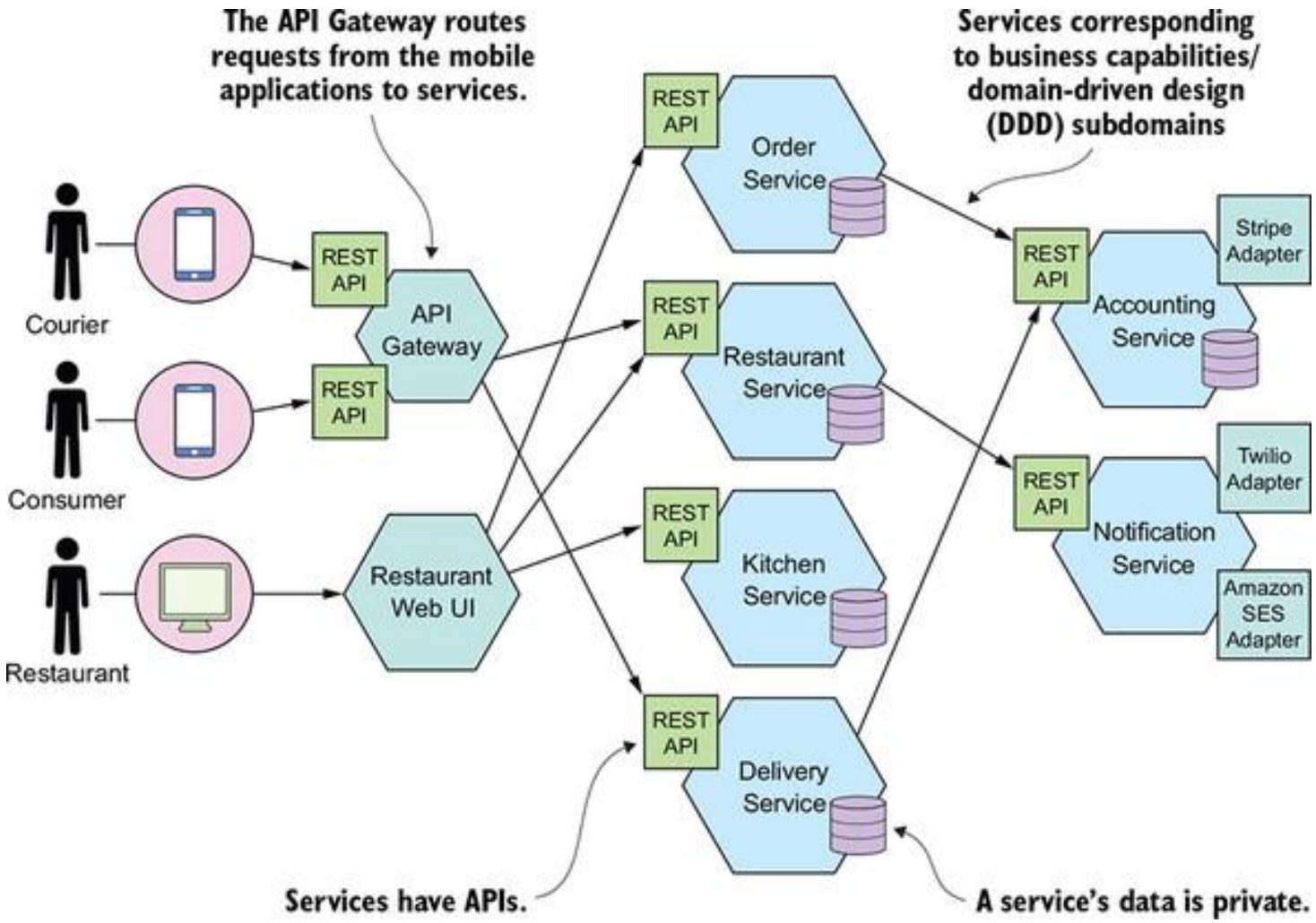
Monolithic architecture (prons&cons)

- Monolithic architecture
  - Prons
    - Simple to develop & easy to make radical changes
    - Straightforward to test & deploy
    - Easy to scale
      - only horizontal scaling – scalability can be limited by a shared database – all the application functionalities are replicated
  - Cons
    - When the complexity increase (and technologies evolve) development, testing, deployment and scaling became much more difficult



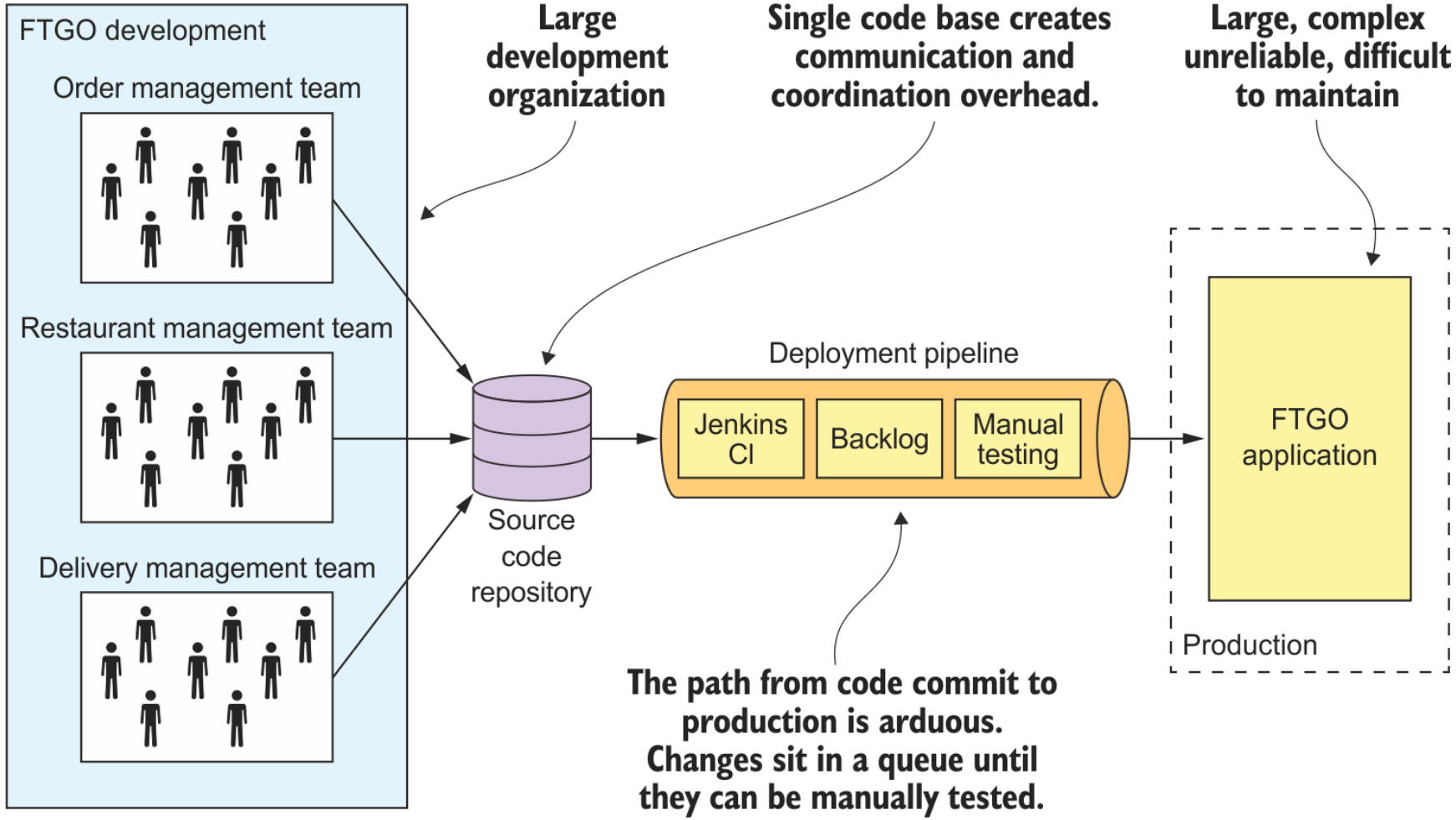
Monolithic vs Microservices





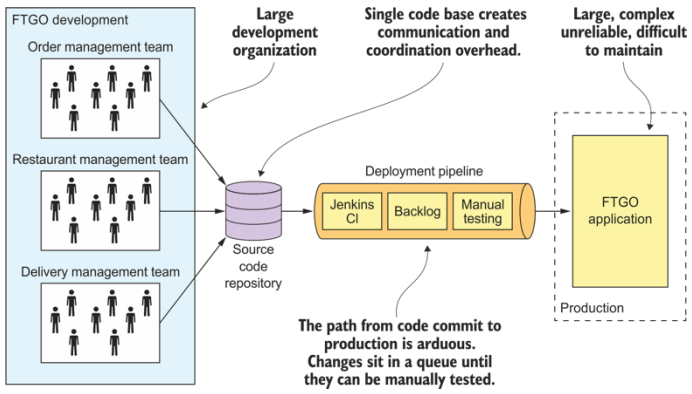
From microservices patterns

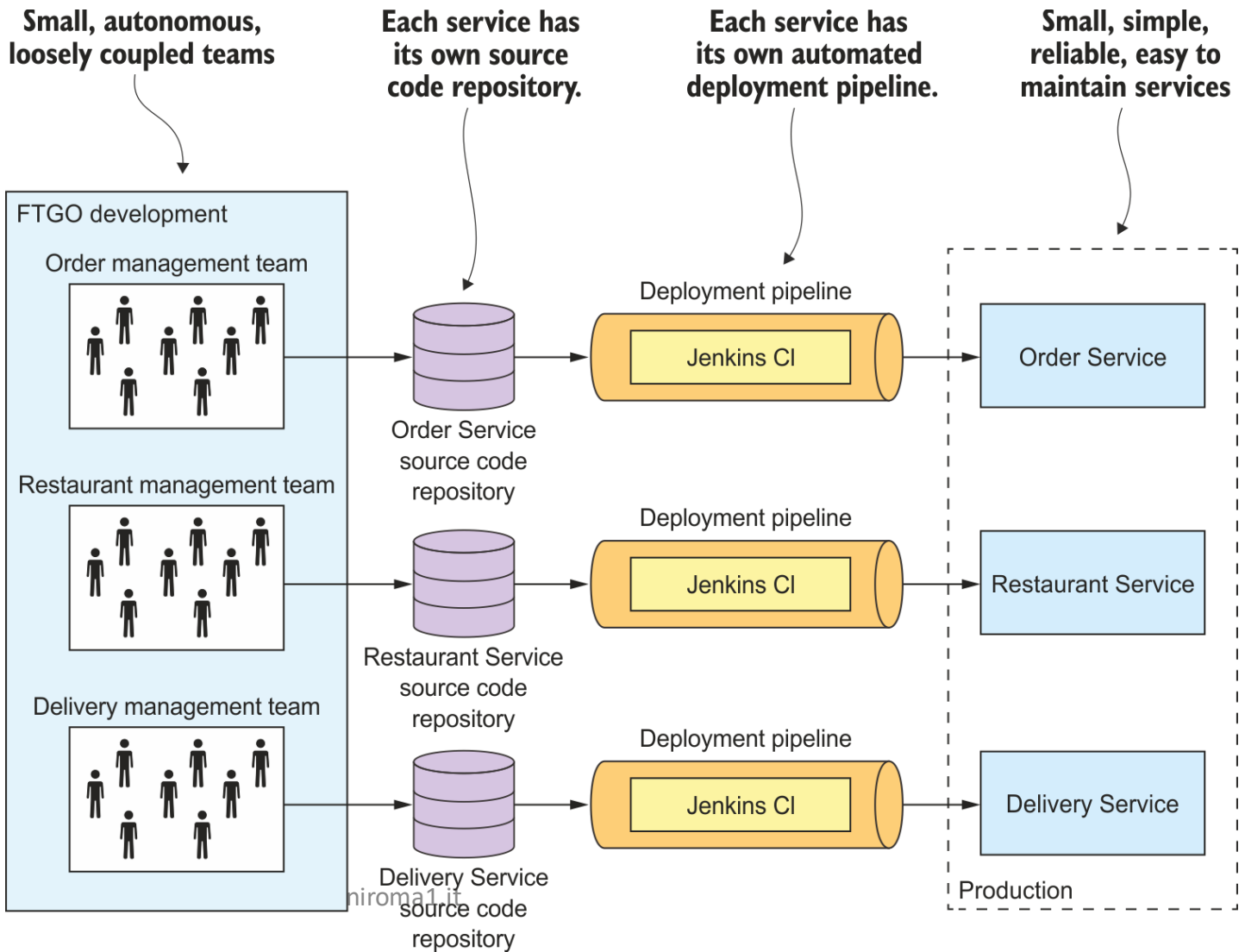
Monolithic vs Microservices (from code commit to production)



From microservices patterns

Monolithic vs Microservices (from code commit to production)





From microservices patterns

What are the differences between SOA and Microservices?

- Service Characteristics
  - Taxonomy
  - Ownership and Coordination
  - Granularity
- Architecture Characteristics
  - Component Sharing
  - Service Orchestration and Choreography
  - Middleware and API
  - Access to remote services
- Architecture Capabilities
  - Application scope
  - Heterogeneous interoperability

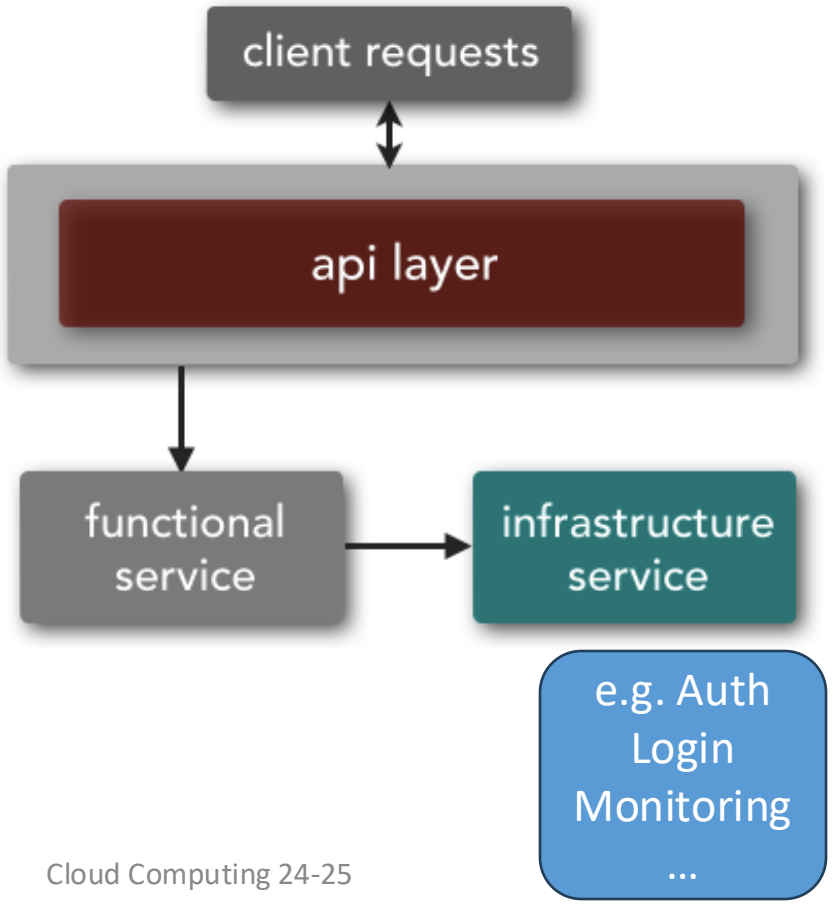
Microservices vs. Service-Oriented Architecture

Mark Richards

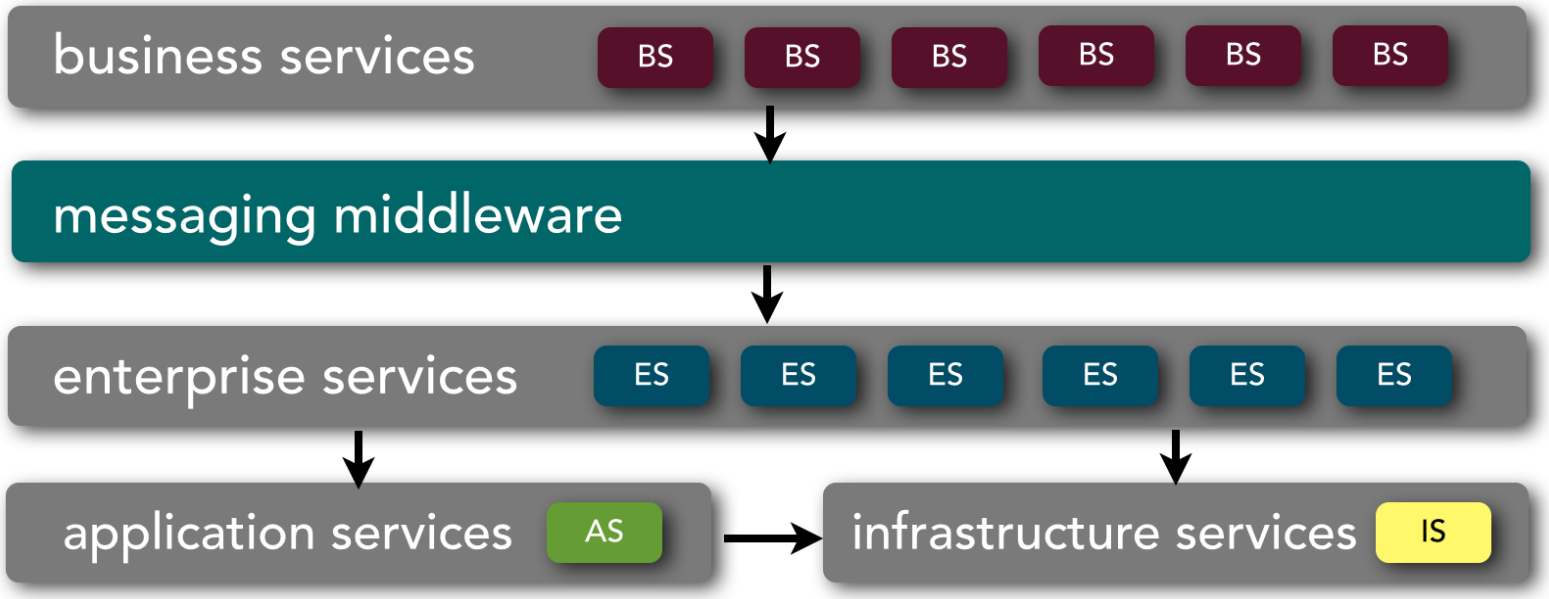
Published by O'Reilly Media, Inc.

SOA vs mS taxonomy

microservice

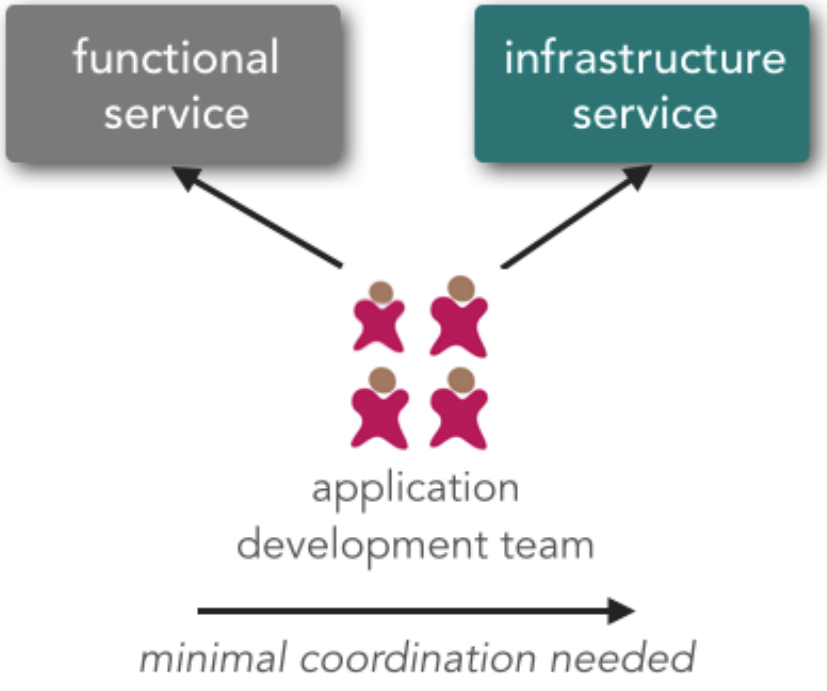


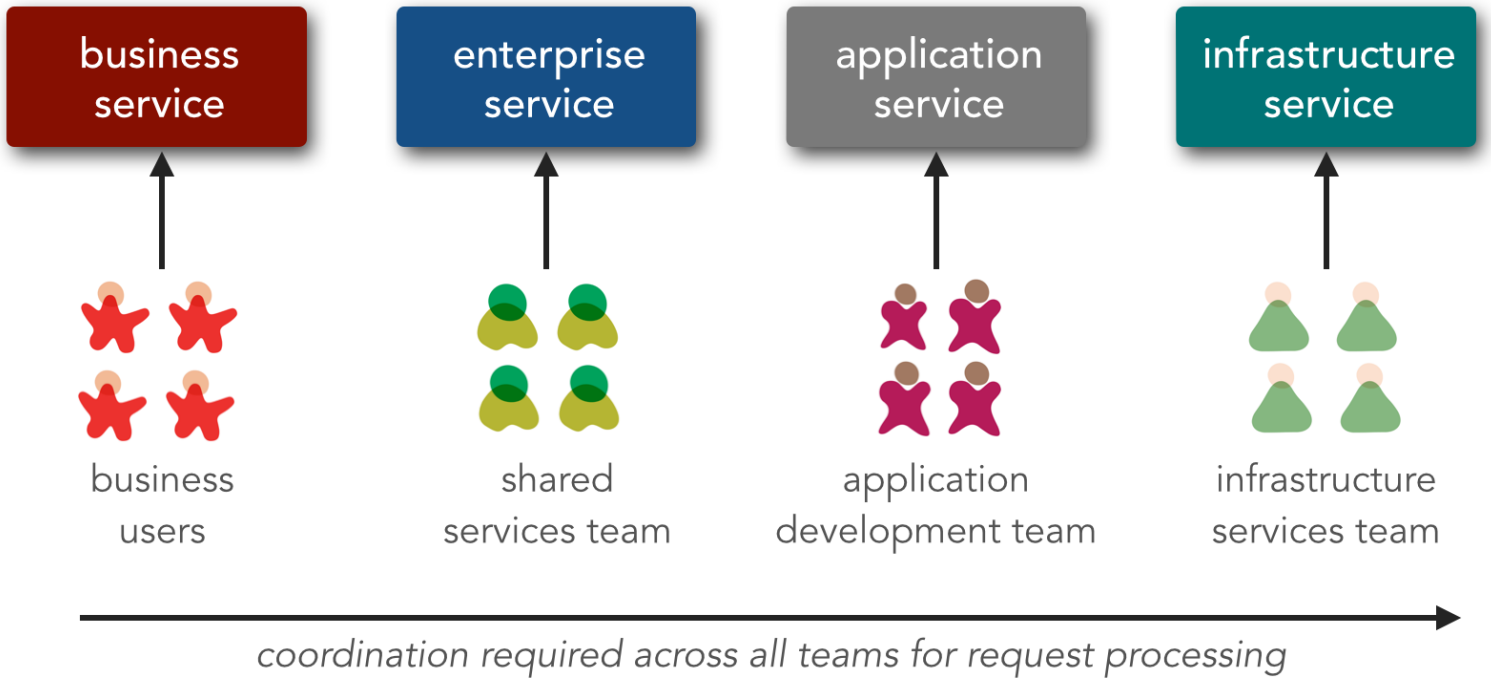
SOA



Service Ownership and Coordination

- A service owner is responsible for creating and maintaining a service.



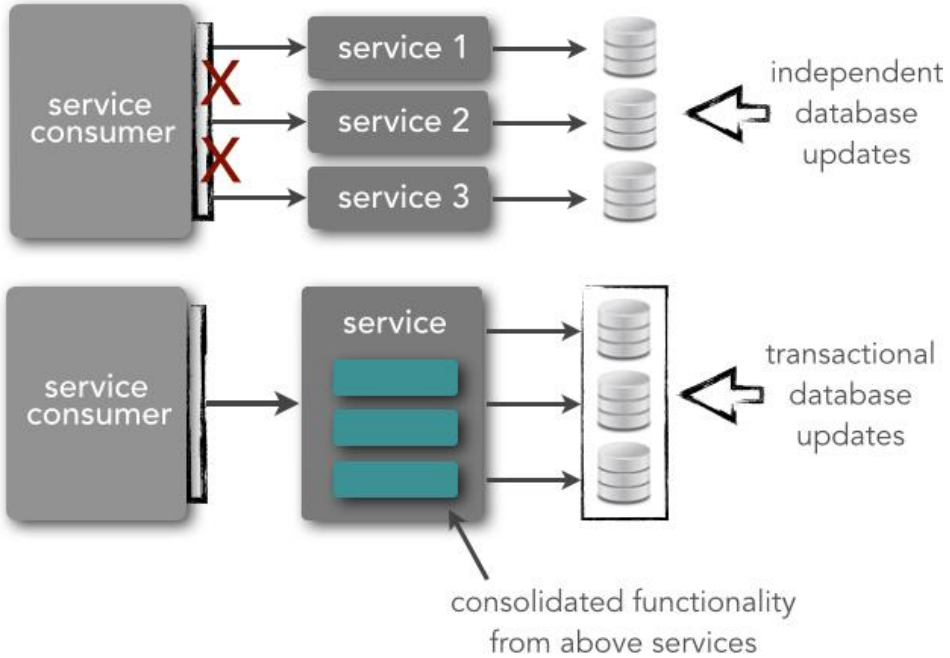


Little of no coordination among teams

High coordination effort among teams

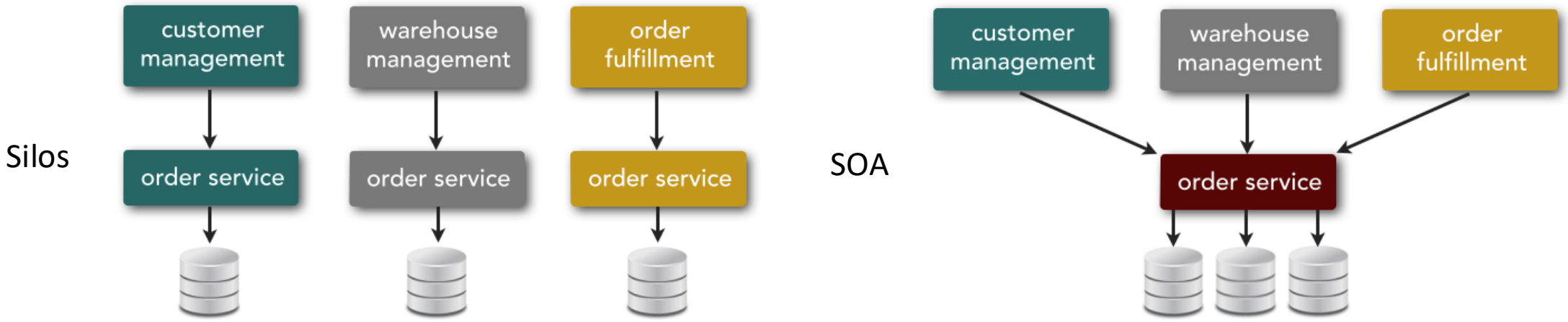
Service granularity

- Service components within a microservices architecture are generally single-purpose (specialized) services
- Service components within a SOA can range in size anywhere from small application services to very large enterprise services
  - it is common to have a service component within SOA represented by a large product or even a subsystem.
- Granularity has impact on
  - Performance
  - Transaction management (ACID/BASE)



Component Sharing

- SOA: **share-as-much-as-possible** architecture style
- microservices architecture: **share-as-little-as-possible** architecture style



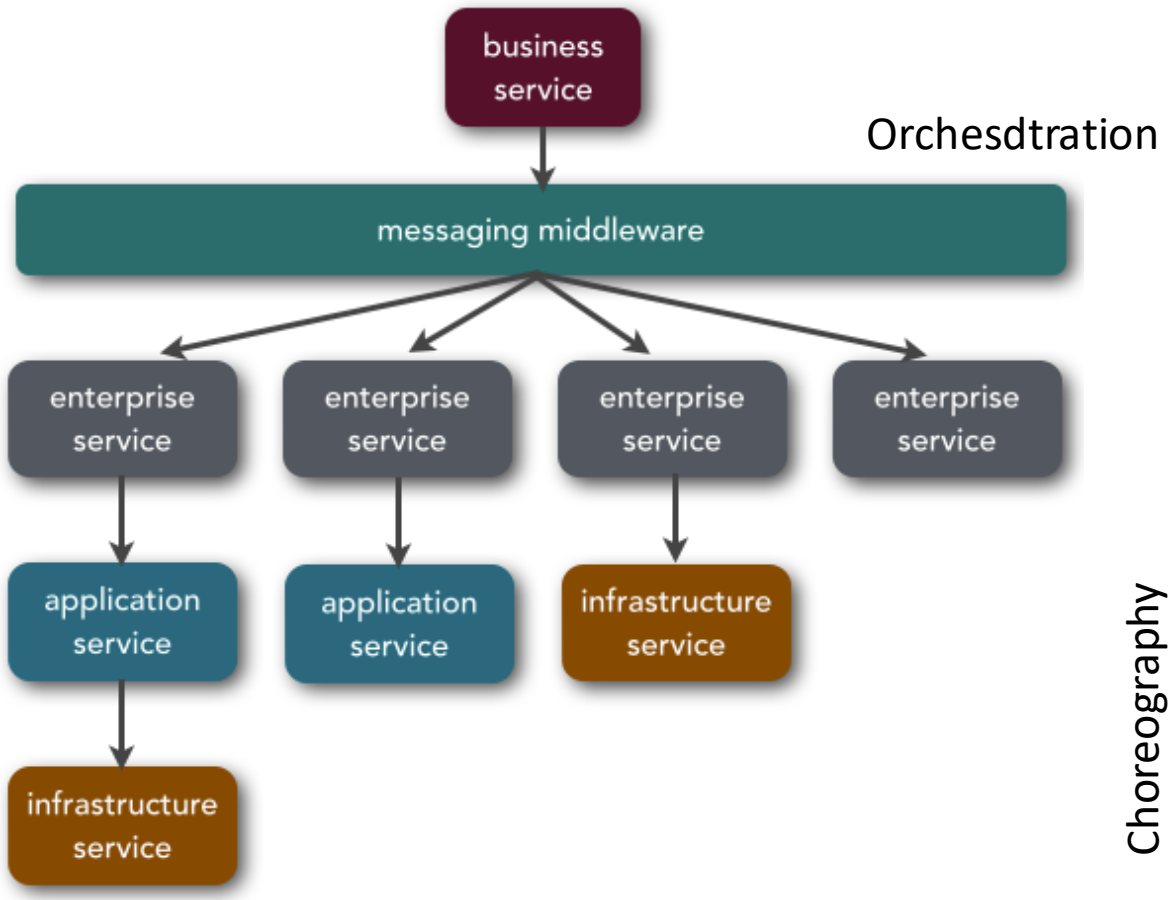
- Share-as-much-as-possible lead to tightly coupled components and increases the overall risk associated with change

Component Sharing

- Microservices architecture leverages a concept called a bounded context
  - A bounded context refers to the coupling of a component (or in this case, a service) and its associated data as a single closed unit with minimal dependencies
- A service component designed this way is
  - self-contained
  - only exposes a well-defined interface and a well-defined contract
  - could violate the Don't-Repeat-Yourself principle
- Advantages
  - Easier service maintenance (change/evolution) because lack of dependent modules
  - Easier service deployment becomes; less code to deploy; less risk that a change made to one module or service will affect other parts of the application
- As result: **more-robust applications that have fewer side effects based on service changes**

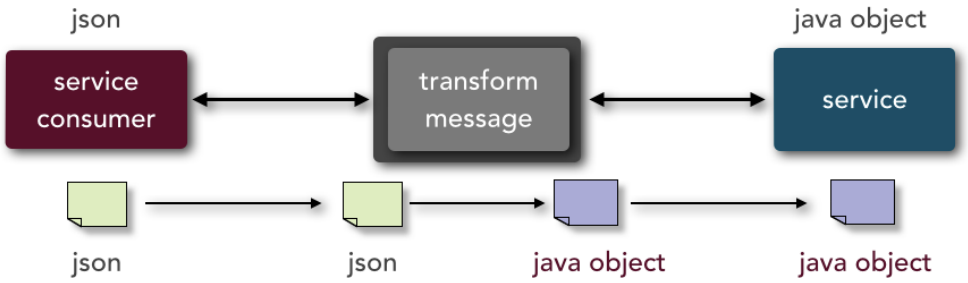
Service Choreography and Orchestration

- Microservices architecture favors service choreography over service orchestration, primarily because the architecture topology lacks a centralized middleware component
  - Too much service choreography leads to **efferent coupling**
- SOA relies on both service orchestration and service choreography



Middleware vs. API Layer

- Microservices
  - does not support the concept of messaging middleware
  - supports the notion of an API layer in front of the services that acts as a service-access facade.
    - it forms an abstraction layer so that service consumers don't need to know the actual location of the service endpoints
    - allows to change the granularity level of a services without impacting the service consumers
- SOA
  - Relies on messaging middleware (or integration bus) to coordinate service calls.
  - The messaging middleware provides additional architectural capabilities e.g.
    - mediation and routing
    - message enhancement
    - message transformation
    - protocol transformation.



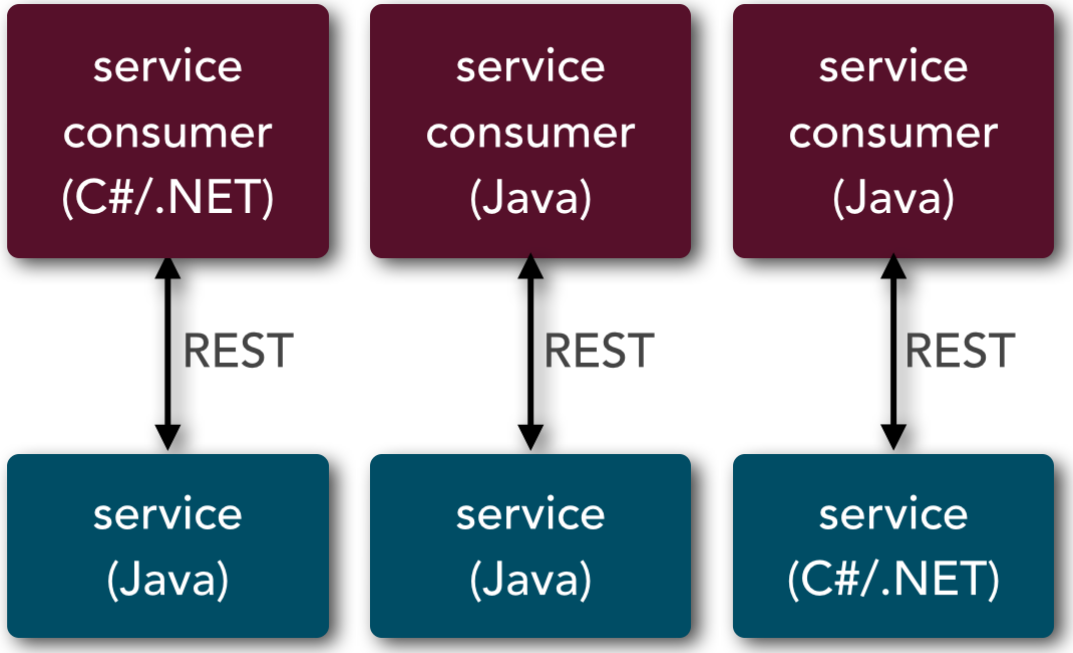
Application Scope

- Application scope refers to the overall size of the application that an architecture pattern can support
- SOA is well-suited for
  - large, complex, enterprise-wide systems that require integration with many heterogeneous applications and services
  - applications that have many shared components, particularly components that are shared across the enterprise
- Microservices architecture is better suited for smaller, well-partitioned web-based systems rather than large-scale enterprise-wide systems.

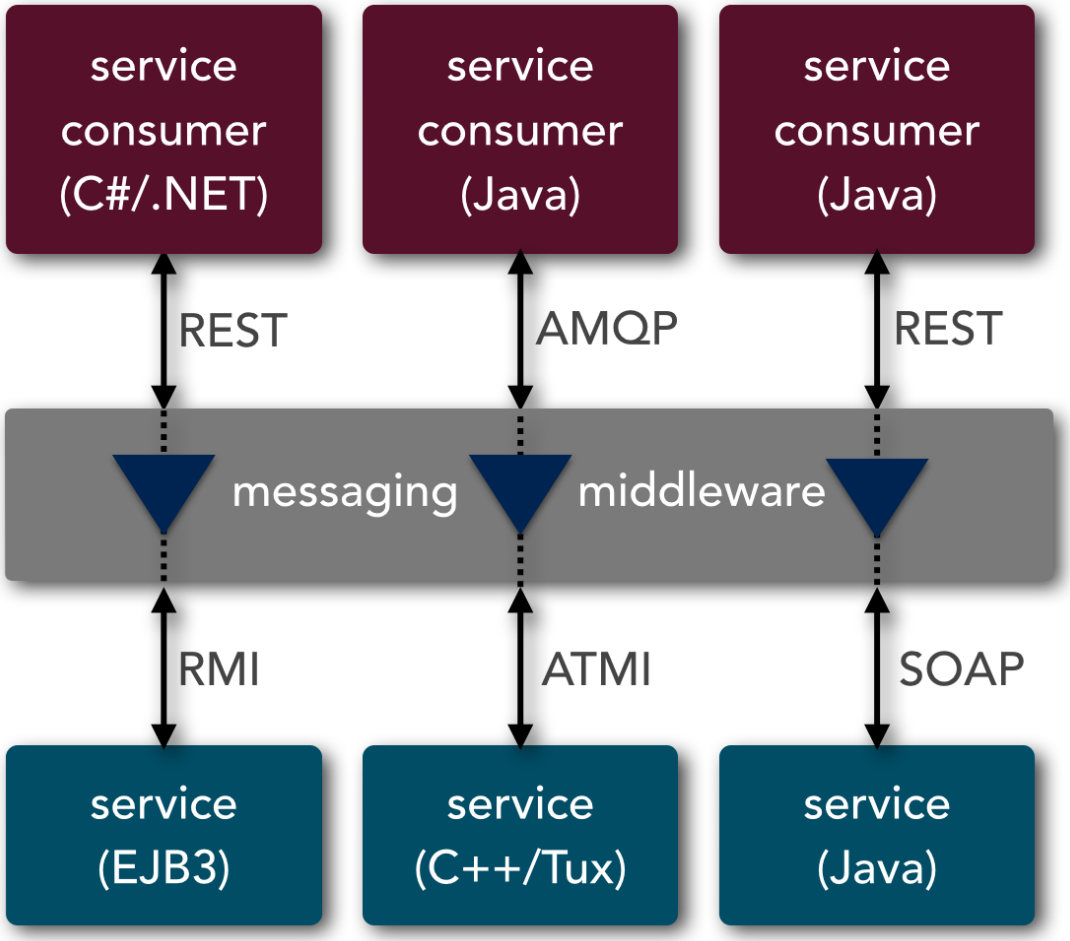
Heterogeneous interoperability

- Heterogeneous interoperability refers to the ability to integrate with systems implemented in different programming languages and platform
- The **microservices architecture** style attempts to simplify the architecture pattern and corresponding implementations by reducing the number of choices for services integration
  - REST and simple messaging are two of the most common remote-access protocols used
- SOA, has no upper limit and promotes the proliferation of multiple heterogeneous protocols through its messaging middleware component.

- Microservices



- SOA



To review the content (some questions)

- Define a distributed system and exmplain why cloud computing is a distributed system.
- What is an Architecture Style? Can you make some example?
- What is a Service Oriented Architecture?
- What is a microservice architecture?
- What are the difference between Orchestration and Choreography? Make examples of Service Orchestration and Choreography
- What are the advantages of using microservices with respect to monolithic architectures?
- What are the main features of microservices
- What are the differences between the SOA and microservice architecture?
- Comment on the benefits and drawbacks of microservices